

AWS Generative AI Interview Preparation Guide

Structured Q&A; — Introductory Level

Amazon Bedrock · Amazon Q Developer · Amazon SageMaker · RAG · AI Agents

Topic	Questions
Amazon Bedrock	5
Amazon Q Developer	5
Amazon SageMaker	5
Retrieval-Augmented Generation (RAG)	5
AI Agents in Bedrock	5
Common Cross-Questions	12

Designed for DevOps Engineers claiming basic exposure to AWS AI/ML services.

■ Amazon Bedrock

Q1. What is Amazon Bedrock?

- Amazon Bedrock is a fully managed AWS service.
- It gives you access to powerful AI models (called Foundation Models) from multiple providers — all through a single, unified API.
- You don't need to set up any servers or manage any infrastructure.
- Providers include Anthropic (Claude), Meta (Llama), Amazon (Nova), Mistral AI, Cohere, and others.
- Think of it as a 'one-stop shop' for using pre-built AI models on AWS.

Q2. What are Foundation Models (FMs)?

- Foundation Models are very large AI models that have already been trained on massive amounts of data.
- Because they are pre-trained, they can perform many tasks without additional training.
- Common tasks they can do: generating text, summarising content, answering questions, translating languages, and writing code.
- Examples: Claude (by Anthropic), Llama (by Meta), Amazon Nova.
- You can use them as-is or fine-tune them with your own data for specific tasks.

Q3. What are the benefits of using Amazon Bedrock instead of hosting models yourself?

- No infrastructure management — AWS handles all the servers and hardware for you.
- Automatic scaling — it scales up or down based on your usage automatically.
- Multiple model choices — you can switch between different AI models from various providers easily.
- Pay-as-you-use pricing — you only pay for what you actually use, no upfront costs.
- Security and compliance — AWS manages security, making it enterprise-ready out of the box.
- Faster development — you can start building AI applications much faster without setup overhead.

Q4. Which model providers are available in Amazon Bedrock?

- Anthropic — Claude models (Claude 3 Sonnet, Haiku, Opus).
- Amazon — Nova models (Amazon's own foundation models).
- Meta — Llama models (open-source large language models).
- Mistral AI — Mistral and Mixtral models.
- Cohere — Command and Embed models (great for search and embeddings).
- AI21 Labs — Jurassic models for text generation.

■ *You don't need to memorise all versions — just know the major providers.*

Q5. What are some common use cases of Amazon Bedrock?

- Chatbots — build conversational AI assistants for customers or internal teams.
- Content Generation — automatically write articles, emails, product descriptions, etc.
- Summarisation — condense long documents, meeting notes, or reports into key points.
- Question Answering — answer questions based on your company's documents or data.
- RAG Applications — combine AI with your private knowledge base for accurate answers.
- Code Generation — generate, explain, or review code snippets using AI.
- Image Generation — create images from text descriptions (with supported models).

■ Amazon Q Developer

Q1. What is Amazon Q Developer?

- Amazon Q Developer is an AI-powered coding assistant built by AWS.

- It helps developers write code faster, understand existing code, fix bugs, and generate documentation.
- It is deeply integrated with AWS — so it also understands AWS services and can guide you on AWS-specific tasks.
- It works inside popular development tools like VS Code and JetBrains IDEs.
- Think of it as a smart coding companion that also knows AWS inside out.

Q2. How can Amazon Q Developer help DevOps Engineers?

- Generate scripts — it can write Bash, Python, or PowerShell scripts for automation tasks.
- Explain AWS services — ask Q to explain what an AWS service does and how to use it.
- Troubleshoot infrastructure — describe an error or issue and Q helps you diagnose and fix it.
- Assist with IaC tools — helps write and review Terraform, CloudFormation, and CDK code.
- Help with Kubernetes — explains Kubernetes manifests and helps write Helm charts.
- CI/CD support — assists with writing pipeline configurations for CodePipeline, GitHub Actions, etc.

Q3. What types of tasks can Amazon Q Developer perform?

- Code Generation — write new functions, scripts, or boilerplate code based on your description.
- Code Explanation — explain what an existing piece of code does in plain English.
- Code Review — identify potential bugs, security issues, or improvements in your code.
- Documentation Generation — auto-generate comments and documentation for code.
- AWS Architecture Guidance — suggest the right AWS services for a given use case.
- Security Scanning — scan code for common vulnerabilities.

Q4. How is Amazon Q different from ChatGPT?

- AWS Integration — Q Developer is deeply integrated with AWS services; ChatGPT is general purpose.
- Context-aware AWS assistance — Q understands your AWS account context (with permissions enabled).
- IDE Integration — Q works natively inside VS Code and JetBrains; ChatGPT requires a browser.
- AWS Account Insights — Q can look at your actual AWS environment and give specific advice.
- Security & Compliance — Q is built to meet AWS enterprise security standards.
- Focused Use Case — Q is optimised for software development and AWS; ChatGPT covers everything.

■ *In interviews: emphasise that Q is AWS-native and developer-focused, not just a generic chatbot.*

Q5. Which development environments support Amazon Q Developer?

- VS Code — one of the most popular code editors, with a Q Developer plugin.
- JetBrains IDEs — IntelliJ IDEA, PyCharm, GoLand, etc.
- AWS Console — Q is available directly in the AWS Management Console.
- AWS Cloud9 — AWS's cloud-based IDE.
- AWS Toolkit — any environment that supports the AWS Toolkit extension.
- Command Line — available via the AWS CLI and terminal for some features.

■ Amazon SageMaker

Q1. What is Amazon SageMaker?

- Amazon SageMaker is a fully managed AWS service for Machine Learning (ML).
- It lets data scientists and ML engineers build, train, and deploy their own custom ML models.
- It covers the entire ML lifecycle — from preparing data to monitoring deployed models.
- It provides notebooks, training jobs, model registries, and deployment infrastructure all in one place.
- Think of it as a complete ML development platform on AWS.

Q2. What are the main stages of an ML workflow in SageMaker?

- 1. Data Preparation — clean, label, and process raw data using SageMaker Data Wrangler or processing jobs.
- 2. Model Training — train your ML model using built-in algorithms or custom code on managed compute.
- 3. Model Evaluation — assess model performance using metrics like accuracy, precision, and recall.
- 4. Deployment — deploy the trained model as a real-time endpoint or batch transform job.
- 5. Monitoring — track model performance over time and detect data drift using SageMaker Model Monitor.

■ Remember the order: Prepare → Train → Evaluate → Deploy → Monitor.

Q3. What is the difference between Amazon SageMaker and Amazon Bedrock?

- SageMaker — used when you want to BUILD and TRAIN your own custom ML model from scratch.
- Bedrock — used when you want to USE already-built foundation models without training anything.
- SageMaker — gives you full control over training data, algorithms, and model architecture.
- Bedrock — gives you quick access to powerful AI models via a simple API call.
- SageMaker — better for traditional ML tasks (predictions, classification, regression).
- Bedrock — better for Generative AI tasks (text, images, chatbots, summarisation).

■ Simple rule: Build your own model → SageMaker. Use a pre-built model → Bedrock.

Q4. What is a SageMaker Endpoint?

- A SageMaker Endpoint is a managed, scalable API endpoint for serving ML model predictions.
 - Once your model is trained, you deploy it to an endpoint so applications can call it in real-time.
 - It automatically handles load balancing, scaling, and availability.
 - You send data to the endpoint, and it returns a prediction or inference result.
 - Example: an endpoint that accepts a customer review as input and returns a sentiment score.
-

Q5. When would you choose SageMaker over Bedrock?

- Custom model training — when you need to train a model on your own specific dataset.
 - Traditional ML use cases — classification, regression, anomaly detection, recommendation systems.
 - Full ML lifecycle management — when you need to manage the entire pipeline from data to deployment.
 - Custom algorithms — when you want to use your own ML algorithm or framework (TensorFlow, PyTorch).
 - Cost optimisation for large-scale training — when running heavy training jobs at scale.
 - Compliance requirements — when you need complete control over model training data and process.
-

■ Retrieval-Augmented Generation (RAG)

Q1. What is Retrieval-Augmented Generation (RAG)?

- RAG is a technique that improves AI responses by fetching relevant information before generating an answer.
 - Instead of relying only on what the AI model was trained on, it first searches an external knowledge source.
 - The retrieved information is then given to the AI model as additional context to generate a more accurate answer.
 - This is especially useful for answering questions about private, recent, or company-specific information.
 - Simple analogy: RAG is like giving a student access to a textbook before answering an exam question.
-

Q2. Why is RAG used?

- Reduce hallucinations — AI models sometimes make up answers; RAG grounds responses in real documents.
 - Improve accuracy — answers are based on actual retrieved data, not just model memory.
 - Use private/company data — you can integrate your organisation's internal documents into the AI.
 - Keep information up to date — the knowledge base can be updated without retraining the AI model.
 - Cost-effective — updating a knowledge base is cheaper than retraining a large language model.
-

Q3. What are the main components of a RAG system?

- Knowledge Base — the collection of documents, PDFs, or data that the system can search through.
- Embeddings — numerical representations (vectors) of text that capture meaning, used for similarity search.
- Vector Database — a specialised database that stores embeddings and enables fast similarity searches.
- Retriever — the component that searches the vector database to find the most relevant documents.
- Large Language Model (LLM) — the AI model that generates the final answer using the retrieved context.

■ *In Bedrock: Knowledge Bases feature handles embeddings, vector storage, and retrieval automatically.*

Q4. How does a RAG workflow operate?

- Step 1 — User asks a question (e.g., 'What is our leave policy?').
- Step 2 — The question is converted into an embedding (a numerical vector).
- Step 3 — The vector database is searched to find the most similar/relevant documents.
- Step 4 — The retrieved documents are sent to the LLM as additional context.
- Step 5 — The LLM generates a final, accurate answer based on the retrieved context.

■ *The key insight: the LLM doesn't guess — it uses retrieved facts to answer.*

Q5. What are some practical use cases of RAG?

- Internal company chatbots — employees can ask questions answered from HR policies or SOPs.
 - Knowledge bases — customer-facing assistants that answer questions from product documentation.
 - Customer support systems — agents that retrieve answers from support articles automatically.
 - Documentation assistants — help developers navigate large codebases or technical documentation.
 - Legal and compliance tools — search through contracts and regulations to answer specific queries.
 - Healthcare — retrieve relevant medical literature before generating clinical summaries.
-

■ AI Agents in Amazon Bedrock

Q1. What is an AI Agent in Amazon Bedrock?

- A Bedrock Agent is an AI-powered system that can understand a goal and autonomously figure out the steps to achieve it.
 - Unlike a simple chatbot, an Agent can take actions — like calling APIs, querying databases, or running Lambda functions.
 - It can plan, reason, and execute multi-step tasks without human intervention for each step.
 - It is powered by a foundation model but has additional tools and instructions that extend its capabilities.
 - Think of it as an AI assistant that can not only talk but also do things on your behalf.
-

Q2. How is an Agent different from a normal chatbot?

- Chatbot — responds to user input with text. It can only answer questions or have conversations.
 - Agent — can take actions: call APIs, retrieve data, update systems, run code, and chain multiple steps.
 - Chatbot — stateless; each message is usually independent.
 - Agent — maintains context and can execute a plan across multiple steps to complete a task.
 - Example: a chatbot tells you how to create a support ticket; an agent actually creates the ticket for you.
-

Q3. What are the main components of a Bedrock Agent?

- Foundation Model — the AI brain (e.g., Claude) that understands requests and decides what to do.
- Instructions — a system prompt that tells the Agent its role, behaviour, and constraints.
- Knowledge Base — optional; gives the agent access to documents for RAG-based retrieval.
- Action Groups — define what actions the Agent can perform (API calls, Lambda functions, etc.).
- APIs / Lambda Integrations — the actual tools the agent uses to interact with external systems.

■ *An Agent = Brain (FM) + Memory (KB) + Hands (Action Groups).*

Q4. What are Action Groups in Bedrock Agents?

- Action Groups are the 'hands' of a Bedrock Agent — they define what the Agent can actually do.
- Each Action Group maps to one or more API operations or AWS Lambda functions.
- The Agent decides which action to call based on the user's request and its instructions.
- Example actions: create a JIRA ticket, fetch data from a database, send an email, query an API.
- Action Groups are defined using an OpenAPI schema, which tells the agent what parameters are needed.

■ *Without Action Groups, an Agent can only respond with text — it can't take real-world actions.*

Q5. What are some use cases of Bedrock Agents?

- IT support automation — automatically diagnose issues and restart services or create tickets.
 - Ticket management — create, update, and assign support tickets in JIRA or ServiceNow.
 - Employee assistants — answer HR queries and also perform actions like booking leaves.
 - Customer service automation — handle customer requests end-to-end without human agents.
 - Workflow automation — orchestrate multi-step business processes automatically.
 - DevOps automation — trigger deployments, check logs, or scale infrastructure based on alerts.
-

■ Common Cross-Questions Interviewers Ask

■ Bedrock ↔ RAG

Can Bedrock be used to build a RAG application?

Q: Can Bedrock be used to build a RAG application?

→ Yes. Amazon Bedrock has a built-in 'Knowledge Bases' feature that lets you create a RAG pipeline without writing complex code. You upload documents, Bedrock converts them to embeddings, stores them in a vector store, and retrieves them when a user asks a question.

Why is RAG often combined with Bedrock?

Q: Why is RAG often combined with Bedrock?

→ Bedrock's foundation models are powerful but only know what they were trained on. RAG adds your private or up-to-date data on top, making responses more accurate and relevant to your specific use case.

Does Bedrock store company documents automatically?

Q: Does Bedrock store company documents automatically?

→ No. You need to explicitly set up a Knowledge Base in Bedrock, upload your documents, and choose a vector store (like OpenSearch or Amazon Aurora). Bedrock then indexes and retrieves from them.

■ Bedrock ↔ Agents

Can Bedrock Agents use RAG?

Q: Can Bedrock Agents use RAG?

→ Yes. You can attach a Knowledge Base to a Bedrock Agent. When a user asks a question, the Agent can retrieve relevant documents from the knowledge base before generating an answer.

How does an Agent access a knowledge base?

Q: How does an Agent access a knowledge base?

→ The Knowledge Base is linked to the Agent during configuration. When the Agent determines that a knowledge lookup is needed, it automatically queries the vector store and uses the results as context.

Why use an Agent instead of a simple prompt?

Q: Why use an Agent instead of a simple prompt?

→ A simple prompt only gets a text response. An Agent can take real actions — call APIs, retrieve data from databases, trigger workflows — making it capable of completing end-to-end tasks, not just answering questions.

■ SageMaker ↔ Bedrock

Which service for custom model training vs quick GenAI?

Q: Which service would you choose for custom model training?

→ Amazon SageMaker — it gives you full control to train, tune, and deploy your own ML model using your own data and algorithms.

Q: Which service would you choose for quick GenAI development?

→ Amazon Bedrock — you get immediate access to powerful foundation models via API, with no training required.

Can SageMaker and Bedrock be used together?

Q: Can SageMaker and Bedrock be used together?

→ Yes. You can fine-tune a foundation model using SageMaker and then deploy it, or use SageMaker to pre-process and prepare data that is later used in a Bedrock Knowledge Base for RAG applications.

■ Amazon Q Developer

Using Amazon Q in your DevOps workflow

Q: How would you use Amazon Q in your DevOps workflow?
→ I would use Q Developer in my IDE to generate Terraform or CloudFormation scripts, explain AWS service configurations, troubleshoot deployment errors, and get AWS architecture recommendations — saving significant time on day-to-day tasks.

Can Q help generate Terraform code?

Q: Can Q help generate Terraform code?
→ Yes. You can describe the infrastructure you need in plain English, and Q Developer will generate Terraform HCL code. It can also explain existing Terraform files and suggest improvements.

Can Q explain Kubernetes manifests?

Q: Can Q explain Kubernetes manifests?
→ Yes. You can paste a Kubernetes YAML manifest and ask Q Developer to explain what it does, identify misconfigurations, or suggest best practices — very useful for DevOps engineers working with EKS or Kubernetes clusters.

■ Quick Reference Comparison

Service	Primary Purpose	Best For	Requires Training?
Amazon Bedrock	Access pre-built Foundation Models	GenAI apps, chatbots, RAG	No
Amazon SageMaker	Build & train custom ML models	Custom ML, full lifecycle	Yes
Amazon Q Developer	AI coding & AWS assistant	DevOps, development, IaC	No
Bedrock Knowledge Bases	RAG pipeline setup	Private data Q&A	No
Bedrock Agents	Autonomous AI task execution	Automation, multi-step tasks	No

This guide covers introductory-level AWS GenAI interview preparation. Confidently answering these questions is sufficient for DevOps roles with basic AI/ML exposure. For MLOps-focused roles, deeper SageMaker and model fine-tuning knowledge may be expected.